SLE-3: Architectural Design (Full C4 Model)

Course 02AML204 – Introduction to Artificial Intelligence

PRN 25UAM048

Name Rajvardhan Pravin Patil

Division :A

Link:

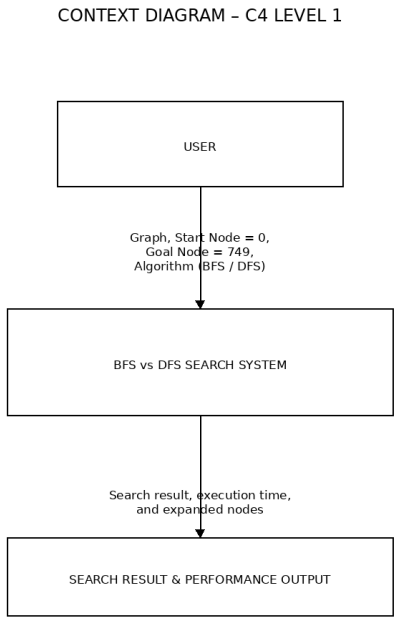
https://github.com/rajpatil31648/SLE3_BFS_DFS_25UAM048.git

1. System Title & Short Description:

BFS vs DFS Search System is a Python-based graph search system that implements Breadth First Search (BFS) and Depth First Search (DFS). The system searches for a goal node from a given start node in the same graph. BFS uses a queue, while DFS uses a stack for exploring nodes. The system also measures execution time and counts expanded nodes for performance comparison.

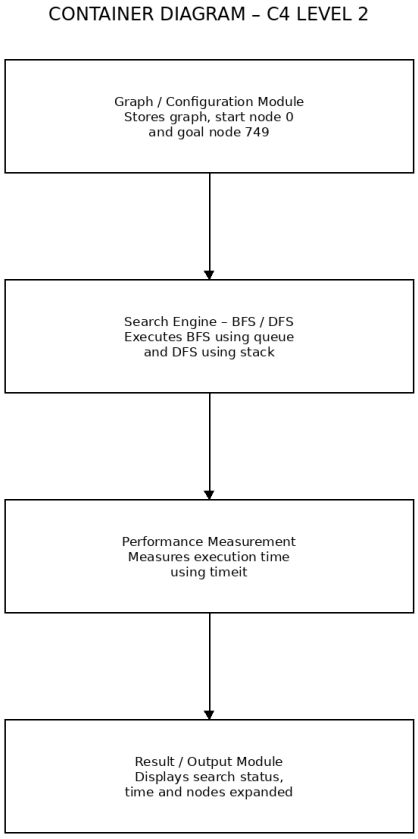
2. Context Diagram (C4 Level 1):

The user provides the search input and starts the BFS or DFS search. The system processes the graph using the selected search algorithm. The system searches from start node 0 to goal node 749. The output provides the search result and performance information.



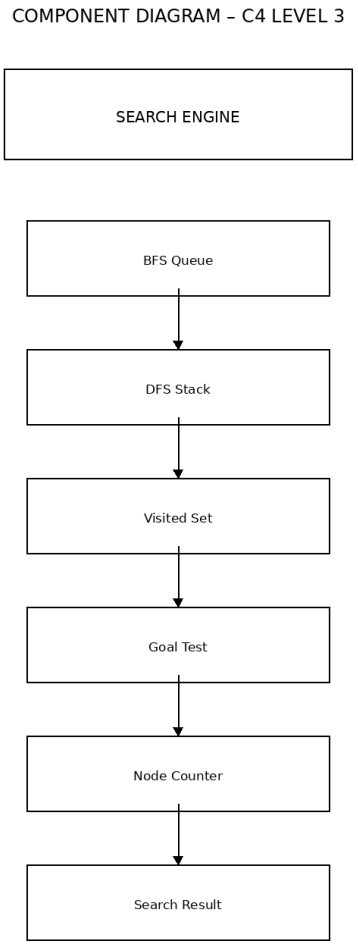
3. Container Diagram (C4 Level 2):

Graph / Configuration Module: Stores the graph and search configuration such as start node 0 and goal node 749.
Search Engine: Executes BFS and DFS algorithms on the same graph using their respective data structures.
Performance Measurement: Measures the execution time of the search algorithms using Python's timeit module.
Result / Output Module: Displays the search status, execution time and number of expanded nodes.



4. Component Diagram (C4 Level 3):

The Search Engine is the main container responsible for graph traversal. It contains BFS and DFS components. BFS uses a queue and DFS uses a stack. The visited set prevents repeated exploration, Goal Test checks whether the goal node is reached, and Node Counter records the number of expanded nodes.



5. Code Level Overview (C4 Level 4):

Main Class / Function	Responsibility
graph / configuration	Defines the graph and search settings: start node 0 and goal node 749.
bfs()	Performs Breadth First Search using a queue.
dfs()	Performs Depth First Search using a stack.
timeit.default_timer()	Measures the execution time of the search.
Node Counter	Counts the number of expanded nodes.
print()	Displays the search and performance results.

6. Design Decisions:

The same graph, start node and goal node are used for both BFS and DFS to maintain a consistent search setup. The Search Engine is the main container because it contains the core search operations. Queue and stack are separate components because BFS and DFS use different search strategies.

7. AI Contribution Note:

AI tools used: ChatGPT. What AI helped with: Understanding the C4 model, organizing the architecture and preparing the diagram structure. What I did myself: I used my SLE-2 BFS/DFS system, understood the code structure and prepared the final architecture documentation.

8. Conclusion:

This SLE helped me understand the architecture of the BFS and DFS search system using the C4 model. The Context level shows overall system interaction, while Container and Component levels show its internal structure. The Code level identifies the main functions used. The C4 model makes the system easier to understand and explain.

SLE-2 reference: The tested graph contains 750 nodes, with start node 0 and goal node 749. BFS expanded 173 nodes and DFS expanded 750 nodes; DFS had the lower measured execution time.